



Vivekanand Education Society's Institute Of Technology
Department Of Information Technology
DSA mini Project
A.Y. 2024-25

Title: Hospital Management System.

Sustainability Goal: Good Health and Well Being

Mentor Name: Mrs. Kajal Jewani.

Domain: Management System

Group Members:

Member1 :Aarya Thorat(61)

Member2: Nikhil Sunchu (59)

1 NO
POVERTY



2 ZERO
HUNGER



3 GOOD HEALTH
AND WELL-BEING



4 QUALITY
EDUCATION



5 GENDER
EQUALITY



6 CLEAN WATER
AND SANITATION



7 AFFORDABLE AND
CLEAN ENERGY



8 DECENT WORK AND
ECONOMIC GROWTH



9 INDUSTRY, INNOVATION
AND INFRASTRUCTURE



10 REDUCED
INEQUALITIES



11 SUSTAINABLE CITIES
AND COMMUNITIES



THE GLOBAL GOALS

For Sustainable Development

12 RESPONSIBLE
CONSUMPTION
AND PRODUCTION



13 CLIMATE
ACTION



14 LIFE BELOW
WATER



15 LIFE
ON LAND



16 PEACE AND JUSTICE
STRONG INSTITUTIONS



17 PARTNERSHIPS
FOR THE GOALS





Content

1. Introduction to the Project
2. Problem Statement
3. Objectives of the Project
4. Scope of the Project
5. Requirements of the System (Hardware, Software)
6. ER Diagram of the Proposed System
7. Data Structure & Concepts Used
8. Algorithm Explanation
9. Time and Space Complexity
10. Front End
11. Implementation
12. Gantt Chart
13. Test Cases
14. Challenges and Solutions
15. Future Scope
16. Code
17. Output Screenshots
18. Conclusion
19. References (in IEEE Format)



Introduction to Project

- Modernizes healthcare operations with automation
- Manages patient records, appointments, and billing
- Built using data structures and algorithms in C
- Enhances patient care and streamlines workflows





Problem Statement

Many hospitals still rely on manual processes for managing patient information, appointments, and medical records, leading to inefficiencies, data redundancy, and increased chances of errors. This lack of automation results in longer wait times for patients and miscommunication among staff. The absence of a centralized, digital solution hampers effective decision-making and resource allocation. Therefore, there is a pressing need for an integrated online hospital management system to address these challenges and improve overall healthcare delivery.



Objectives of the project

1. **User-Friendly Interface:** Create an intuitive platform for patients and staff.
2. **Centralized Data Management:** Develop a robust database for patient records and appointments.
3. **Efficient Scheduling:** Implement a module to optimize appointment scheduling.
4. **Data Security:** Ensure compliance with data protection regulations through encryption.
5. **Real-Time Updates:** Facilitate immediate access to patient information for improved care.
6. **Reporting and Analytics:** Enable report generation for informed decision-making.
7. **Scalability:** Design for future growth and system enhancements.



Requirements of the system (Hardware, software)

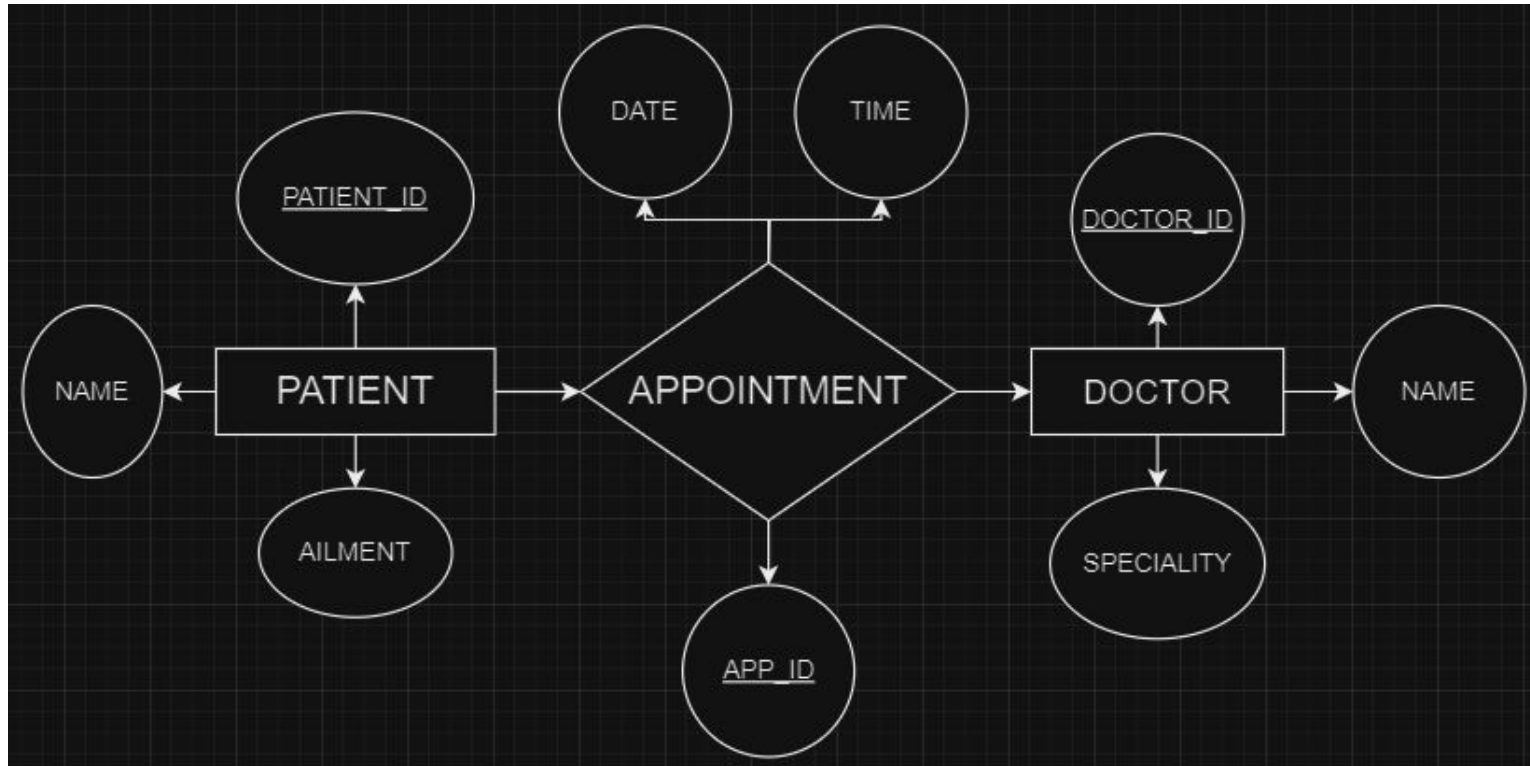
Hardware Requirements:

- Minimum: Dual-core processor, 4 GB RAM, 100 GB HDD.
- Recommended: Quad-core processor, 8 GB RAM, SSD storage.

Software Requirements:

- C programming environment (e.g., GCC).
- Database management system (e.g., MySQL).
- Operating System: Windows/Linux.

ER diagram of the proposed system





Front End

Hospital Management System

Patient Management

Add Patient

Patient ID:

Name:

Ailment:

Add Patient

Search Patient

Search by Name:

Search Patient

Search Patient

Patient Found: ID: 1234, Name: Aarya Santosh Thorat, Ailment: ACL injury

Delete Patient

Delete by Patient ID:

Delete Patient

Display All Patients

No patients registered.

Appointment Management

Schedule Appointment

Process Appointment

No appointments to process.

H
Patient Management
Add Patient
Patient ID:
Enter patient ID
Name:
Enter patient name
Ailment:
Enter patient ailment
Add Patient

Snipping Tool



Data Structures And Concepts Used

1. Linked List: A linked list is used to manage the patients in the system. Each node in the linked list represents a patient with details such as Patient ID, Name, and Ailment. The linked list allows dynamic addition and deletion of patients.
2. Queue: A queue is used to manage appointments in the system. The queue follows a First-Come-First-Served (FCFS) approach, where the first appointment scheduled will be the first one to be processed. The queue has two pointers: front: Points to the first appointment in the queue. rear: Points to the last appointment in the queue.
3. Structs: Used to define custom data types like Patient and Appointment to store patient details and appointment information.
4. Pointers: Pointers are used extensively to manage memory addresses for linked lists and queues.
5. Dynamic Memory Allocation



Algorithm

1. **Initialize:**
 - Create an empty list for patients.
 - Create an empty queue for appointments.
2. **Display Menu:** Show the user options to add, search, delete, display patients, schedule appointments, process appointments, or exit.
3. **User Input:** Take the user's choice between 1-6.
4. **Perform Actions Based on Input:**
 - **Add Patient:** Input patient ID, name, ailment, and add to the linked list.
 - **Search Patient:** Input patient name, search in the list, and display details if found.
 - **Delete Patient:** Input patient ID, search, and remove from the list.
 - **Display Patients:** Traverse the linked list and print patient details.
 - **Schedule Appointment:** Input patient ID, search for the patient, and add an appointment to the queue.
 - **Process Appointment:** Remove and display the first patient in the queue.
5. **Exit:** Terminate the program if the exit option is selected.



Time and Space Complexity

Time Complexity

1. **createPatient:** $O(1)$
2. **addPatient:** $O(n)$ (traverse list to end)
3. **searchPatient:** $O(n)$ (searching by name)
4. **deletePatient:** $O(n)$ (search and delete by ID)
5. **displayPatients:** $O(n)$ (traverse the list)
6. **createQueue:** $O(1)$
7. **enqueue:** $O(1)$
8. **dequeue:** $O(1)$

Space Complexity

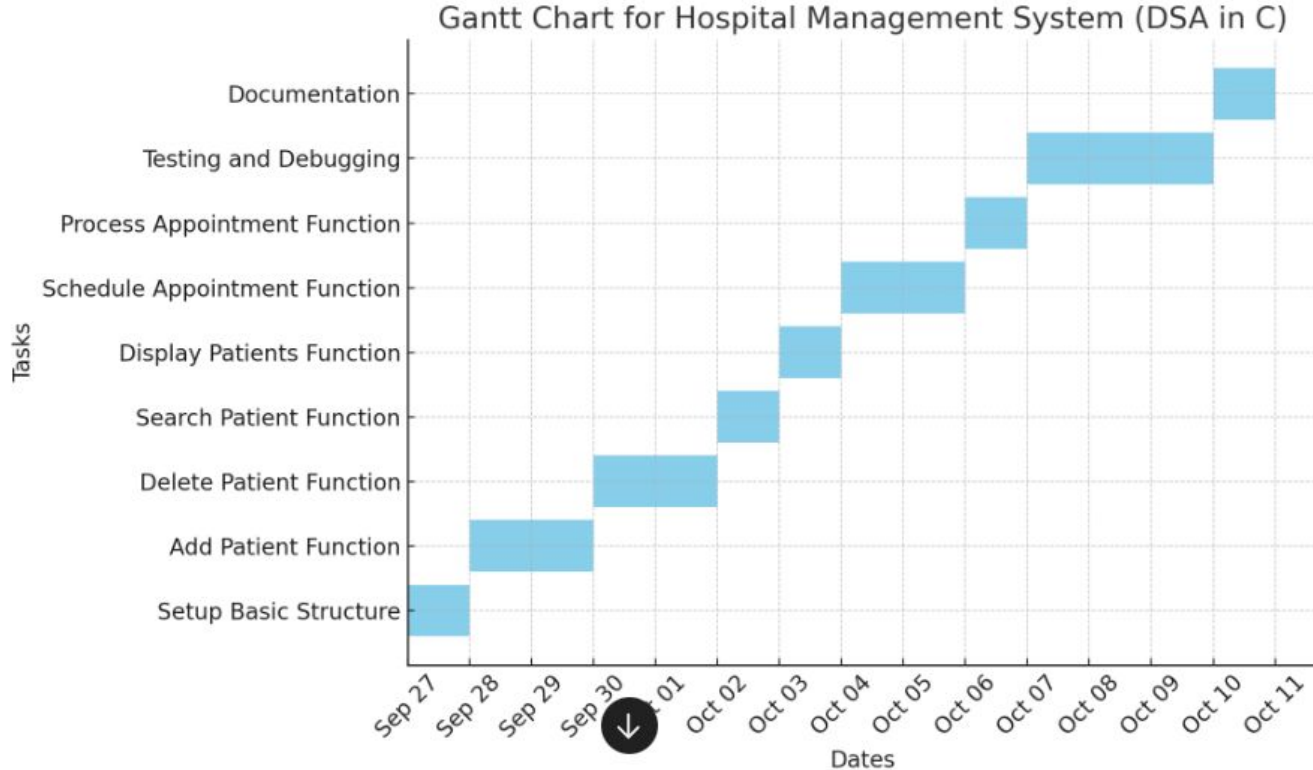
1. **createPatient:** $O(1)$
2. **addPatient:** $O(1)$ (new node added)
3. **searchPatient:** $O(1)$
4. **deletePatient:** $O(1)$
5. **displayPatients:** $O(1)$
6. **createQueue:** $O(1)$
7. **enqueue:** $O(1)$ (new node added)
8. **dequeue:** $O(1)$



Implementation

1. `createPatient(int id, char name[], char ailment[])`: Creates and returns a new patient node with the provided ID, name, and ailment.
2. `addPatient(Patient* head, int id, char name[], char ailment[])`: Adds a new patient to the linked list (at the end) and returns the updated list.
3. `searchPatient(Patient* head, char name[])`: Searches for a patient by name in the linked list. Returns the patient node if found, otherwise returns `NULL`.
4. `deletePatient(Patient* head, int id)`: Deletes a patient by ID from the linked list and returns the updated list.
5. `displayPatients(Patient* head)`: Displays all patients' details (ID, name, and ailment) in the linked list.
6. `createQueue()`: Creates and returns a new queue for managing appointments.
7. `enqueue(Queue* q, int patientID, char patientName[])`: Adds a new appointment to the end of the queue (based on patient ID and name).
8. `dequeue(Queue* q)`: Removes and processes the first appointment (from the front of the queue), printing the details.

Gantt Chart





Test Cases

1. **Add Patient:**
 - Input: ID=123, Name="Aarya"
 - Output: Patient added.
2. **Search Patient:**
 - Input: Name="Aarya"
 - Output: Patient found.
3. **Delete Patient:**
 - Input: ID=123
 - Output: Patient deleted.
4. **Display Patients:**
 - Input: N/A
 - Output: List of patients displayed.
5. **Schedule Appointment:**
 - Input: ID=123
 - Output: Appointment scheduled.
6. **Process Appointment:**
 - Input: N/A
 - Output: Appointment processed.
7. **Edge Case (Empty List/Queue):**
 - Input: Delete or display with no patients.
 - Output: "No patients registered."



Challenges and Solution

1. Memory Management

- **Challenge:** Memory leaks from `malloc`.
- **Solution:** Ensure every `malloc` has a corresponding `free`.

2. Concurrent Access

- **Challenge:** Multiple users modifying data simultaneously.
- **Solution:** Use locking mechanisms for data integrity.

3. Input Validation

- **Challenge:** Invalid user input causing errors.
- **Solution:** Implement robust validation checks.

4. Scalability

- **Challenge:** Inefficient management with many patients.
- **Solution:** Use efficient data structures like hash tables.

5. User Experience

- **Challenge:** Confusing command-line interface.
- **Solution:** Design a clear menu and consider a GUI.

6. Data Integrity

- **Challenge:** Maintaining accurate patient records.
- **Solution:** Validate data before any modification.



Future Scope

1. **Graphical User Interface (GUI):** Develop a user-friendly interface for better accessibility.
2. **Database Integration:** Use databases (e.g., MySQL) for efficient data management.
3. **Mobile Application:** Create an app for patients to manage appointments and access records.
4. **Telemedicine Features:** Incorporate virtual consultations and remote monitoring.
5. **AI and Machine Learning:** Use AI for predictive analytics and personalized treatment.



Code

```
Patient* createPatient(int id, char name[], char ailment[]) {
    Patient* newPatient = (Patient*)malloc(sizeof(Patient));
    newPatient->id = id;
    strcpy(newPatient->name, name);
    strcpy(newPatient->ailment, ailment);
    newPatient->next = NULL;
    return newPatient;
}

// insert a patient into the Linked List
Patient* addPatient(Patient* head, int id, char name[], char ailment[]) {
    Patient* newPatient = createPatient(id, name, ailment);
    if (head == NULL) {
        return newPatient;
    }
    Patient* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newPatient;
    return head;
}
```

```
Patient* searchPatient(Patient* head, char name[]) {
    Patient* temp = head;
    while (temp != NULL) {
        if (strcmp(temp->name, name) == 0) {
            return temp;
        }
        temp = temp->next;
    }
    return NULL;
}

Patient* deletePatient(Patient* head, int id) {
    if (head == NULL) return NULL;

    if (head->id == id) {
        Patient* temp = head;
        head = head->next;
        free(temp);
        return head;
    }

    Patient* current = head;
    while (current->next != NULL && current->next->id != id) {
        current = current->next;
    }

    if (current->next == NULL) {
        printf("Patient not found.\n");
        return head;
    }

    Patient* temp = current->next;
    current->next = current->next->next;
    free(temp);
}
```



Output SS

```
--- Hospital Management System ---
```

1. Add Patient
2. Search Patient
3. Delete Patient
4. Display Patients
5. Schedule Appointment
6. Process Appointment
7. Exit

```
Enter your choice: 1
```

```
Enter Patient ID: 1234
```

```
Enter Patient Name: AARYA
```

```
Enter Ailment: Fracture
```

```
--- Hospital Management System ---
```

1. Add Patient
2. Search Patient
3. Delete Patient
4. Display Patients
5. Schedule Appointment
6. Process Appointment
7. Exit

```
Enter your choice: 2
```

```
Enter Patient Name to Search: AARYA
```

```
Patient Found: ID: 1234, Name: AARYA, Ailment: Fracture
```

```
--- Hospital Management System ---
```

1. Add Patient
2. Search Patient
3. Delete Patient
4. Display Patients
5. Schedule Appointment
6. Process Appointment
7. Exit

```
Enter your choice: 3
```

```
Enter Patient ID to Delete: 1234
```

```
--- Hospital Management System ---
```

1. Add Patient
2. Search Patient
3. Delete Patient
4. Display Patients
5. Schedule Appointment
6. Process Appointment
7. Exit

```
Enter your choice: 5
```

```
Enter Patient ID for Appointment: 12345
```

```
Patient not found, please register first.
```



Conclusion

The Hospital Management System developed in C effectively streamlines healthcare operations by integrating essential functionalities such as patient registration, search, removal, and appointment management.

Key benefits include:

- 1. Improved Patient Management:** Quick access to patient records enhances personalized care and treatment outcomes.
- 2. Efficient Appointment Scheduling:** Reduces scheduling conflicts, optimizing healthcare resource utilization.
- 3. Data Integrity and Security:** Structured data storage ensures accurate and secure management of patient information.
- 4. User-Friendly Interface:** The straightforward menu makes navigation easy for healthcare professionals.
- 5. Scalability:** The system can be expanded to include additional features like billing and advanced reporting.

In summary, this Hospital Management System not only meets current administrative needs but also positions healthcare facilities for future improvements in patient care and operational efficiency, ultimately leading to better patient outcomes and satisfaction.



References

IEEE REFERENCE PAPERS:

1. **A Study on the Construction and Management of Cloud-based Patients Accessible Hospital Information Systems.**
R. Wang, D. Gu and F. Tao, "A study on the construction and management of cloud-based patients accessible hospital information systems," 2015 International Conference on Computer and Computational Sciences (ICCCS), Greater Noida, India, 2015, pp. 198-202, doi: 10.1109/ICCACS.2015.7361349. keywords: {Hospitals;Cloud computing;Medical diagnostic imaging;Information systems;Information technology;Government;Hospital information system;technical level;organization environment;doctor-patient relationship;hospital information management},
2. **Knowledge Management Integration in Hospital Management Information System (SIMRS) Implementation in XYZ Hospital.**
N. Fitriani, R. F. Rachmadi and P. H. Mukti, "Knowledge Management Integration in Hospital Management Information System (SIMRS) Implementation in XYZ Hospital," 2023 IEEE 7th International Conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE), Purwokerto, Indonesia, 2023, pp. 191-196, doi: 10.1109/ICITISEE58992.2023.10404400. keywords: {Electrical engineering;Hospitals;Statistical analysis;Collaboration;Management information systems;Knowledge management;Information technology;Hospital;Knowledge Management;descriptive statistical analysis;SIMRS},
3. **Research on the Logistics Management Service Socialization of Public Hospitals in Yunnan Province.**
Zheng Jiliang, Zhang Ya and Wang Yuntian, "Research on the logistics management service socialization of public hospitals in Yunnan province," 2016 13th International Conference on Service Systems and Service Management (ICSSSM), Kunming, 2016, pp. 1-4, doi: 10.1109/ICSSSM.2016.7538558. keywords: {Logistics;Hospitals;Identity management systems;logistics management;logistics service socialization;public hospital;social service},